

Pointers
Strings
Loops
Complex and Nested Loops
Control Flows

1. Output of this program?

```
```cpp
#include <iostream>
using namespace std;
int main()
{
 int i, j, var = 'A';
 for (i = 5; i >= 1; i--) {
 for (j = 0; j < i; j++)
 printf("%c ", (var + j));
 printf("\n");
 }
 return 0;
}
```
```

****Output:****

```
```
A B C D E
A B C D
A B C
A B
A
```
```

2. Output of the program?

```
```c
#include <stdio.h>
void f(char**);
int main()
{
 char *argv[] = { "ab", "cd", "ef", "gh", "ij", "kl" };
 f(argv);
 return 0;
}
void f(char **p)
{
 char *t;
 t = (p += sizeof(int))[-1];
 printf("%s\n", t);
}
```
```

****Output:****

```
```
gh
```
```

3. Output of the following program?

```
```c
#include <stdio.h>
void dynamic(int s, ...)
{
 printf("%d ", s);
}
int main()
{
 dynamic(2, 4, 6, 8);
}
```

```

 dynamic(3, 6, 9);
 return 0;
}
...
```

**\*\*Output:\*\***  
 ...

2 3  
 ...

### 4. Predict the output?

```

...c
#include <stdio.h>
int main()
{
 void demo();
 void (*fun)();
 fun = demo;
 (*fun)();
 fun();
 return 0;
}
void demo()
{
 printf("program ");
}
...
```

**\*\*Output:\*\***  
 ...

program program  
 ...

### 5. Explain: What is the output of the following program?

```

...c
#include <stdio.h>
int main()
{
 char str1[] = "ZohoInterview";
 char str2[] = {'t', 'e', 's', 't', 't', 'e', 's', 't', 't', '1'};
 int n1 = sizeof(str1)/sizeof(str1[0]);
 int n2 = sizeof(str2)/sizeof(str2[0]);
 printf("n1 = %d, n2 = %d", n1, n2);
 return 0;
}
...
```

**\*\*Output:\*\***  
 ...

n1 = 14, n2 = 9  
 ...

**\*\*Explanation:\*\***

- `str1` is a null-terminated string, so its length is 14 (13 characters + 1 null terminator).
- `str2` is an array of characters without a null terminator, so its length is 9.

### 6. Explain: Predict the Output

```

...c
#include <stdio.h>
int main()
{
 char str[] = "Aptitude";
```

```

 printf("%s %s %s\n", &str[5], &5[str], str+5);
 printf("%c %c %c\n", *(str+6), str[6], 6[str]);
 return 0;
}
...

```

**\*\*Output:\*\***

```

...
ude ude ude
d d d
...

```

**\*\*Explanation:\*\***

- `&str[5]`, `&5[str]`, and `str+5` all point to the 6th character of `str`, which is `'u'`.
- `*(str+6)`, `str[6]`, and `6[str]` all access the 7th character of `str`, which is `'u'`.

### 7. Explain: Predict the output of the below program

```

...c
#include <stdio.h>
#define SIZE(arr) sizeof(arr) / sizeof(*arr)
void fun(int* arr, int n)
{
 int i;
 *arr += *(arr + n - 1) += 10;
}
void printArr(int* arr, int n)
{
 int i;
 for(i = 0; i < n; ++i)
 printf("%d ", arr[i]);
}
int main()
{
 int arr[] = {10, 20, 30};
 int size = SIZE(arr);
 fun(arr, size);
 printArr(arr, size);
 return 0;
}
...

```

**\*\*Output:\*\***

```

...
50 20 40
...

```

**\*\*Explanation:\*\***

- `*arr` (10) is increased by the value of `*(arr + n - 1)` (30) plus 10, resulting in 50.
- `*(arr + n - 1)` (30) is increased by 10, resulting in 40.
- The final array is `{50, 20, 40}`.

### 8. Explain: Print the output

```

...c
#include <stdio.h>
struct st
{
 int x;
 struct st next;
};
int main()
{

```

```

 struct st temp;
 temp.x = 10;
 temp.next = temp;
 printf("%d", temp.next.x);
 return 0;
}

```

**\*\*Output:\*\***

- This code will not compile because `struct st` contains a field of its own type, which is not allowed in C.

### 9. Explain: Identify the output of the following

```

```c
union test
{
    int x;
    char arr[8];
    int y;
};
int main()
{
    printf("%d", sizeof(union test));
    return 0;
}

```

****Output:****

```

8

```

****Explanation:****

- The size of the union is determined by the size of its largest member. `arr[8]` is the largest member, occupying 8 bytes.

10. Output of the program

```

```c
#include <stdio.h>
char *c[] = {"GeksQuiz", "MCQ", "TEST", "QUIZ"};
char **cp[] = {c+3, c+2, c+1, c};
char ***cpp = cp;
int main()
{
 printf("%s ", **++cpp);
 printf("%s ", *--*++cpp+3);
 printf("%s ", *cpp[-2]+3);
 printf("%s ", cpp[-1][-1]+1);
 return 0;
}

```

**\*\*Output:\*\***

```

TEST sQuiz Z CQ

```

**\*\*Explanation:\*\***

- `\*\*++cpp` points to `"TEST"`.  
 - `\*--\*++cpp+3` points to `"Z"`.  
 - `\*cpp[-2]+3` points to `"ST"`.  
 - `cpp[-1][-1]+1` points to `"eksQuiz"`.

11. **\*\*Question 1:\*\***

```

```cpp
#include <iostream>
using namespace std;

int a = 20;

int main() {
    int a = 10;
    std::cout << a << " " << ::a;
    return 0;
}
```

```

**\*\*Output:\*\* 1020**

12. **\*\*Question 2:\*\***

```

```cpp
#include<iostream>

void func(int *b){
    *b = 1;
}

int main() {
    int *a;
    int n;
    a = &n;
    *a = 0;
    func(a);
    std::cout << *a << std::endl;
    return 0;
}
```

```

**\*\*Output:\*\* 1**

13. **\*\*Question 3:\*\***

```

```cpp
#include <iostream>
using namespace std;

class Test {
    static int i;
    int j;
};

int Test::i;

int main() {
    cout << sizeof(Test);
    return 0;
}
```

```

**\*\*Output:\*\* 4**

14. **\*\*Question 4:\*\***

```

```c
#include <stdio.h>

int main() {
    int i;
    char ch[] = {'z', 'o', 'h', 'o'};
    char *ptr, *str1;
    ptr = ch;
    str1 = ch;
    i = (*ptr-- + ++*str1) - 10;
}
```

```

```

 printf("%d", i);
 return 0;
}

```

**\*\*Output:\*\* 236**

15. **\*\*Question 5:\*\***

```

```c
int main() {
    int a[10][10] = {{1,2},{3,4},{5,6},{7,8},{9,10}};
    int *p = a[3];
    int result = (*p + 2) * a[4][1] + (++*p) + (*p + 7);
    printf("%d", result);
    return 0;
}

```

****Output:** 113**

16. ****Question 6:****

```

```c
#include <stdio.h>

void main() {
 int n = 21;
 int out = 1;
 while(n > 3) {
 n /= 2;
 out *= (n / 2);
 }
 printf("%d", n * out);
}

```

**\*\*Output:\*\* 20**

17. **\*\*Question 7:\*\***

```

```c
#include <stdio.h>

int main() {
    int n[] = {8,1,3,9,4};
    int j, *y = n;
    for(j = 0; j < 5; j++) {
        if(j % 2 == 0)
            *y++;
    }
    printf("%d", *y);
    return 0;
}

```

****Output:** 9**

18. ****Question 8:****

```

```c
#include <stdio.h>

int main() {
 int x = 10;
 int y = 20;
 if(!(x ^ y))
 printf("0");
 else
 printf("1");
 return 0;
}

```

```
```
```

```
**Output:** 1
```

```
19. **Question 9:**
```

```
```c
```

```
#include <stdio.h>
```

```
int main() {
 int a[10][10] = {{1,2},{3,4},{5,6},{7,8},{9,10}};
 int *p = a[3];
 int result = (*p + 2) * a[4][1] + (++*p) + (*p + 13);
 printf("%d", result);
 return 0;
}
```

```
```  
**Output:** 119
```

```
20. **Question 10:**
```

```
```c
```

```
#include <stdio.h>
```

```
int calc(int x, int *py, int **ppz) {
 int y, z;
 **ppz += 1;
 z = **ppz;
 *py += **ppz;
 y = *py;
 x += *py;
 return x + y + z;
}
```

```
void main() {
 int c, *b, **a;
 c = 5;
 b = &c;
 a = &b;
 printf("%d", calc(c, b, a));
}
```

```
```  
**Output:** 35
```

```
### 21
```

```
```c
```

```
switch(5)
{
 default:printf("five");
 case 1:printf("one");
}
```

```
```
```

```
Output: `fiveone`
```

Explanation: The `default` case gets executed since there's no matching case for `5`. After `default`, execution falls through to `case 1`, resulting in printing `fiveone`.

```
### 22
```

```
```c
```

```
if(101)
 printf("hai");
else
 printf("all");
```
```

```
Output: `hai`
```

Explanation: `101` is a non-zero value, which is considered `true` in C. Therefore, `printf("hai");` is executed.

```
### 23
```c
for(i=0;i<5;i++);
printf("%d",i);
```
```

Output: `5`

Explanation: The semicolon after the `for` loop creates an empty loop body. After the loop, `i` is `5`, which gets printed.

```
### 24
```c
void main()
{
 int a=10,b=20,c=30;
 printf("%d%d");
}
```
```

Output: Undefined behavior.

Explanation: The `printf` function is missing the variables to be printed, leading to undefined behavior.

```
### 25
```c
main()
{
 int i=0;
 switch(i)
 {
 case 0: i++;
 printf("%d", i);
 case 1: printf("%d", i);
 case 2: printf("%d", i);
 }
}
```
```

Output: `111`

Explanation: `case 0` increments `i` to `1`, then falls through to `case 1` and `case 2`, printing `1`, `1`, `2` in sequence.

```
### 26
```c
#define SQR(x) (x*x)
main()
{
 int a, b=3;
 a = SQR(b+2);
 printf("%d", a);
}
```
```

Output: `11`

Explanation: `SQR(b+2)` expands to `(b+2\*b+2)`, i.e., `3+2\*3+2 = 3+6+2 = 11`.

```
### 27
```c
```

```

int main()
{
 int i;
 int arr[5] = {1};
 for(i = 0; i < 5; i++)
 printf("%d", i);
}
...

```

Output: `01234`

Explanation: The loop prints the values of `i` from `0` to `4`.

```

28
```c
main()
{
    int x=0;
    if(x=0)
        printf("value of x is zero");
    else
        printf("value of x is not zero");
}
...

```

Output: `value of x is not zero`

Explanation: `if(x=0)` assigns `0` to `x`, which is `false`. Thus, `printf("value of x is not zero");` gets executed.

```

### 29
```c
main()
{
 int i=0;
 for(i=0; i<20; i++)
 {
 switch(i)
 {
 case 0: i+=5;
 case 1: i+=2;
 case 5: i+=5;
 default: i+=4;
 break;
 }
 printf("%d", i);
 }
}
...

```

Output: `1621`

Explanation:

- `i = 0`: case 0 adds 5 â†’ `i = 5` falls through, case 1 adds 2 â†’ `i = 7`, case 5 adds 5 â†’ `i = 12`, default adds 4 â†’ `i = 16` then breaks.
- `i = 16`: default adds 4 â†’ `i = 20` then breaks.

```

30
```c
main()
{
    int x=20,y=35;
    x = y++ + x++;
    y = ++y + ++x;
    printf("%d%d",x,y);
}
...

```

Output: `5729`

Explanation:

```
- `x = y++ + x++` â†’ `x = 35 + 20` â†’ `x = 55`, `y = 36`, `x = 21`  
- `y = ++y + ++x` â†’ `y = 37 + 22` â†’ `y = 59`
```

```
### 31  
``c  
void main()  
{  
    printf("\n98");  
    printf("\b76");  
    printf("\r54");  
}  
...
```

Output: `\r54` (results depend on terminal).

Explanation: `\b` moves the cursor back, `\r` moves to the start of the line.

```
### 32  
``c  
int main()  
{  
    int x=3;  
    if(x==2); x=0;  
    if(x==3) x++;  
    else  
        printf("x=%d",x);  
}  
...
```

Output: `1`

Explanation: First `if` does nothing (semicolon); second `if` is false; `else` not executed.

```
### 33  
``c  
int main()  
{  
    printf(" \" hai %% all \" ");  
}  
...
```

Output: `\" hai % all "`

Explanation: Escape sequences print double quotes and percent sign.

```
### 34  
``c  
int main()  
{  
    printf(5+"welcome");  
}  
...
```

Output: `me`

Explanation: Pointer arithmetic prints string starting from the 5th index.

```
### 35  
``c  
int main()  
{  
    printf("%c", 5["welcomeall"]);  
}  
...
```

Output: `m`

Explanation: Equivalent to `printf("%c", "welcomeall"[5]);`.

```
### 36
```c
int main()
{
 int x = printf("welcome");
 printf("%d", x);
}
```
```

Output: `welcome7`

Explanation: `printf("welcome")` returns 7 (number of characters printed).

```
### 37
```c
int main()
{
 printf("%d", printf("%d", 1234));
}
```
```

Output: `12344`

Explanation: Inner `printf` prints `1234` and returns 4, outer `printf` prints `4`.

```
### 38
```c
int main()
{
 int a[25] = {1,2,3,4,5};
 printf("%d", sizeof(a)/sizeof(int));
}
```
```

Output: `25`

Explanation: `sizeof(a)` gives total bytes, divided by `sizeof(int)` gives array length.

```
### 39
```c
void main()
{
 int b=12;
 switch(printf("%c%d", 't', b))
 {
 case 0: printf("Hello"); break;
 case 1: printf("Bye"); break;
 case 5: printf("Welcome"); break;
 default: printf("Hi");
 }
}
```
```

Output: `t12Hi`

Explanation: `printf("%c%d", 't', b)` prints `t12` and returns `3` (3 characters printed). `default` case executes.